CASE STUDY · AI AUTOMATION & TECHNICAL DOCUMENTATION

AI Recruiting Automation Platform

A DeepSeek matching engine and a 14-service communications pipeline, designed and built from scratch, running unattended in production.

AI Automations Expert | Senior Technical Writer | DeepSeek · LLM Workflows

Wessam Mandour

THE PROBLEM

Recruiting top-of-funnel doesn't scale by hand

Hundreds of candidates per role. Multi-channel outreach under provider rate caps. Interview transcripts to score. And a final match a recruiter must actually trust.

Volume

Hundreds of applicants per role, most not a fit, manual screening is hours of work.

Rate limits

Email + WhatsApp outreach must respect provider caps and never double-send.

Transcripts

Screening interviews arrive as raw transcripts that need consistent scoring.

Trust

A bare match score is useless, a recruiter needs to know WHY a candidate ranked.

THE SYSTEM

Two systems, one automated funnel

◆ Matching Engine

Python · FastAPI · DeepSeek

Ranks candidates with a written rationale each, through a six-stage funnel. A full run takes 90 to 120 min on Railway and posts Slack status to the team.

◆ Candidate Flow

Node · Express · 14 services

Webhook-triggered automations (CV parse, transcript scoring) plus cron-scheduled reminder sweeps. 2,000+ WhatsApp and email a day; CV parsed in under a minute.

40,000

rows scanned / run

90-120 min

matching run on Railway

2,000+/day

WhatsApp + email

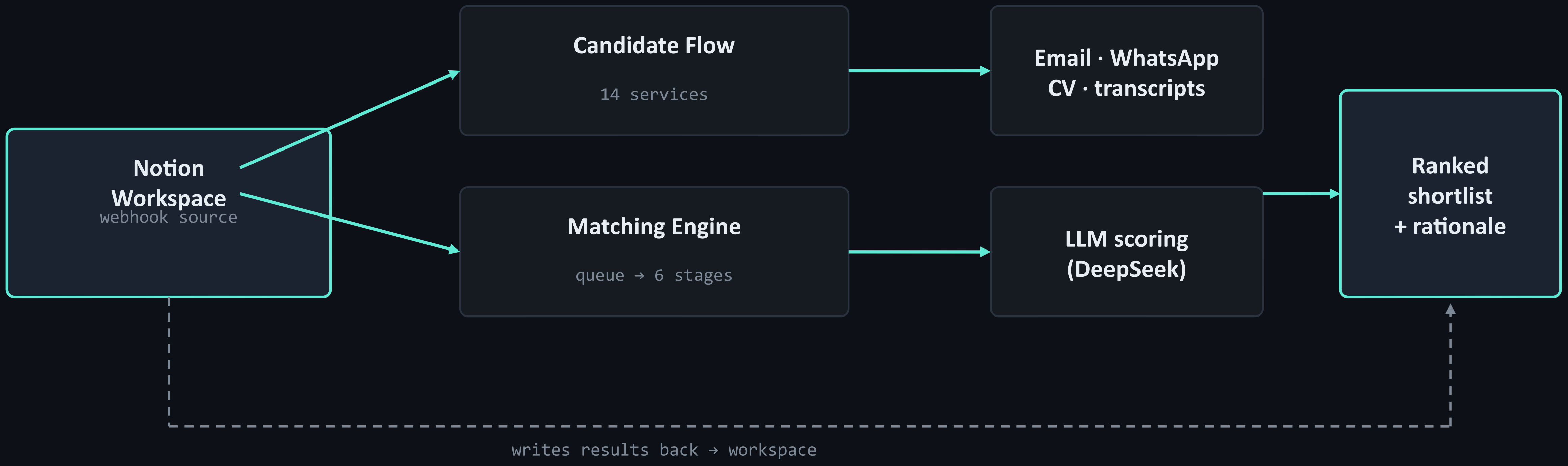
<1 min

CV parsed, regex-first

Deployed on Railway, triggered by webhooks and cron schedules, with Slack status to the whole team.

ARCHITECTURE

Event-driven, with the workspace as source of truth



No shared database to corrupt · failure in one service can't block another · every service stateless & idempotent

DEEP DIVE · MATCHING ENGINE

Six stages: job description → ranked shortlist

Each stage narrows the pool before the next, more expensive stage runs. The LLM only sees candidates worth paying to score.

1

Rubric extraction

LLM turns free-text job spec into a structured rubric: gates, requirements, keywords.

2

Keyword reduction

Cheap local filter drops no-signal candidates BEFORE any paid LLM call, top cost lever.

3

Hard filters

Non-negotiable gates: language, location, minimum screening score.

4

Priority ordering

Bucket by business priority; score the highest-value candidates first.

5

LLM scoring

0-100 + written rationale each, concurrency-capped, retried, failure-isolated.

6

Qualify & backfill

Link qualified first; backfill best below-threshold to hit target, logged.

UNDER THE HOOD

The parts that make it production, not a script

Idempotent re-runs

Scored records skipped on re-run, no double-contact, no duplicate matches.

Rate limiting

Sequential queue, concurrency cap, write delays, under quota by design.

Graceful degradation

Backoff + JSON hardening; one bad candidate can't sink a 200-candidate run.

Schema-drift defense

Missing props ignored, odd types logged & skipped, eligibility re-checked.

AI only where needed

DeepSeek for judgment; regex for parsing, filtering, routing. No wasted tokens.

Observability + Slack

Logs keyed by id, email, stage; Slack status to the team at start and finish.

CONFIGURABLE BY DESIGN

One codebase, many deployments, zero forks

Nothing is hard-coded per client. Behavior is driven entirely by environment variables, so the same service is deployed many times, each tuned to a different goal, by changing config and redeploying.

Same engine, Flash or Pro

One codebase, two deployments: DeepSeek Flash for speed and cost, DeepSeek Pro for tougher roles. A single env var picks the model.

One reminder, deployed 3×

The WhatsApp reminder service runs three times, each with a different "hours since submission" threshold, for a staggered reminder sequence.

Same code, new database

Each branch points at a different Notion database in the same workspace via env vars. New database, new deploy, no code change.

Every client need met by configuration, not custom code, so all deployments stay consistent and safe to change.

TECHNICAL WRITING

The documentation is part of the deliverable

Every service ships an operational handbook. For a system that runs unattended, the docs are the reliability layer.

Audience-layered

One-table overview for managers, branch-level depth for developers, same doc.

Operational

Documents which log line proves which behavior: debug by grep, not by reading source.

Failure-first

Maps real symptoms to the exact gate that caused them.

troubleshooting.md

```
## Troubleshooting

Candidate not considered for a role, check, in order:
  1. Stage-2 keyword overlap
  2. Stage-3 hard filters (language, location, score)
  3. existing scored row (already processed)
  4. duplicate-email de-duplication

Fewer qualified matches than target, expected:
  Stage 6 backfills below-threshold by descending
  score and logs
fallback_selected=true per row.
```

IMPACT

What the automation delivers

40,000

candidate rows scanned per matching run

90-120 min

full run on Railway, with Slack status

2,000+/day

WhatsApp + emails (Twilio + Postmark)

<1 min

to parse a CV, regex-first, AI only when needed

Manual work that took weeks, now seconds to a couple of hours. Workflow on the previous slides.

WHAT THIS DEMONSTRATES

AI automation

14 event-driven services, idempotent, rate-limited, running unattended in production.

Technical writing

Audience-layered operational handbooks, failure-first troubleshooting, this case study.

Cost-aware AI

DeepSeek on judgment-heavy steps, regex everywhere else. No wasted tokens.

Wessam Mandour

wessam.mandour94@gmail.com · linkedin.com/in/wessam-mandour · github.com/Wessam-Mandour